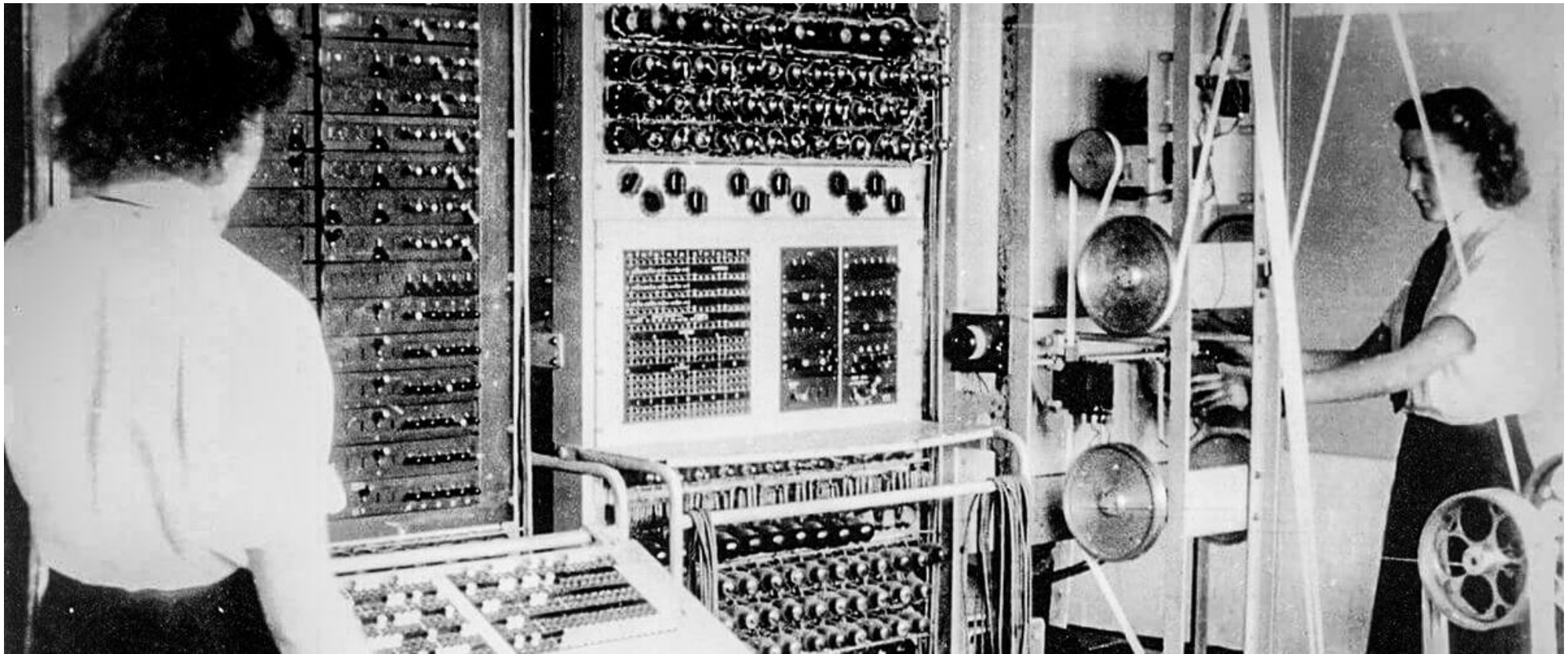
CSC1031: OOP

Week 2

Java Ecosystem

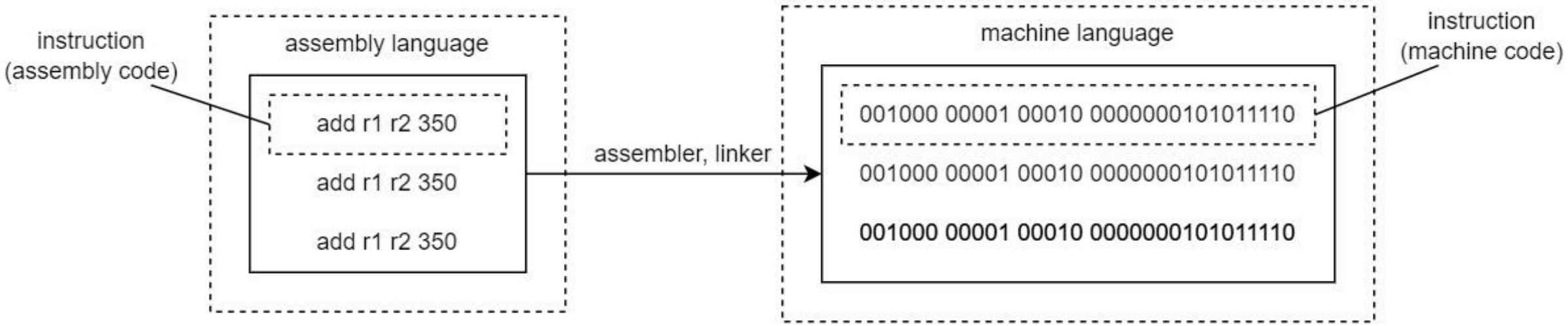
The Evolution of Programming

- **Early Days:** Machine Code
  - Programming with raw binary (0 and 1).
  - Extremely error-prone and difficult to read.
  - No portability: programs tied to specific hardware.



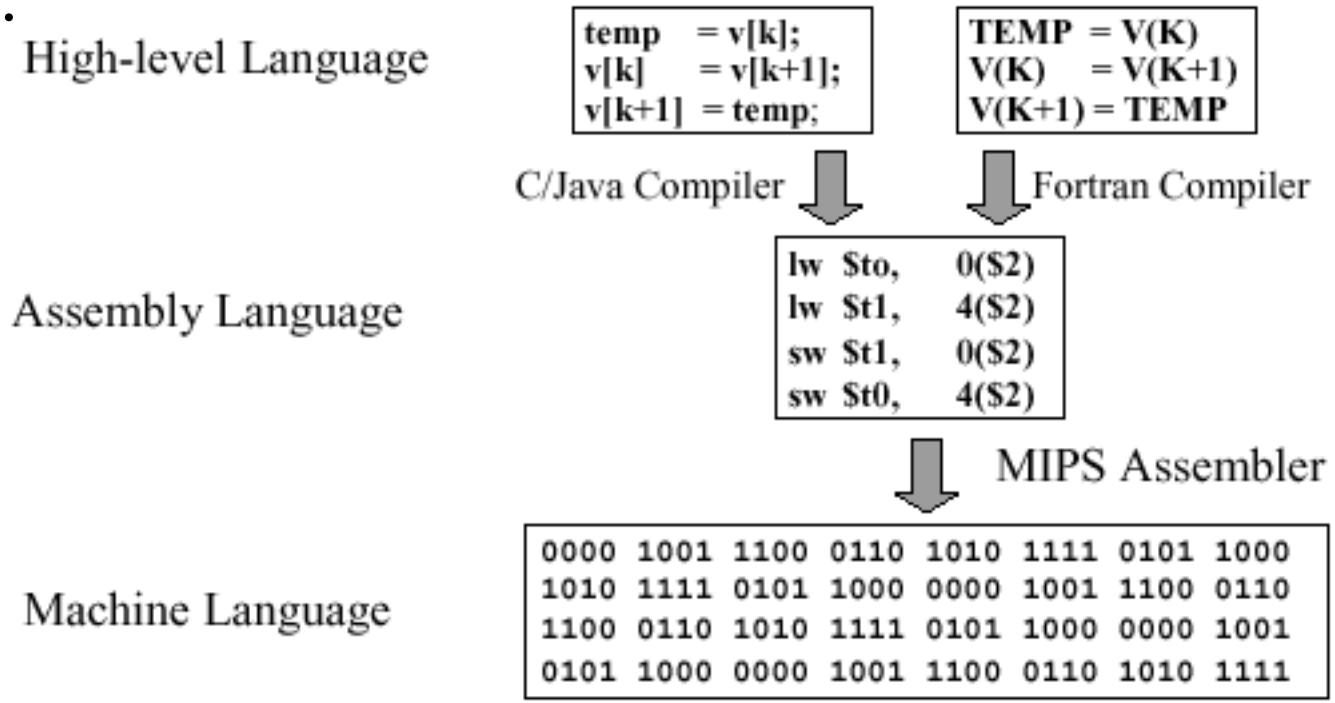
The Evolution of Programming

- **Assembly Language: A Step Forward**
  - Mnemonic codes (ADD, SUB) instead of binary.
  - Easier to understand but still tied to hardware.
  - Required an assembler to translate to machine code.



The Evolution of Programming

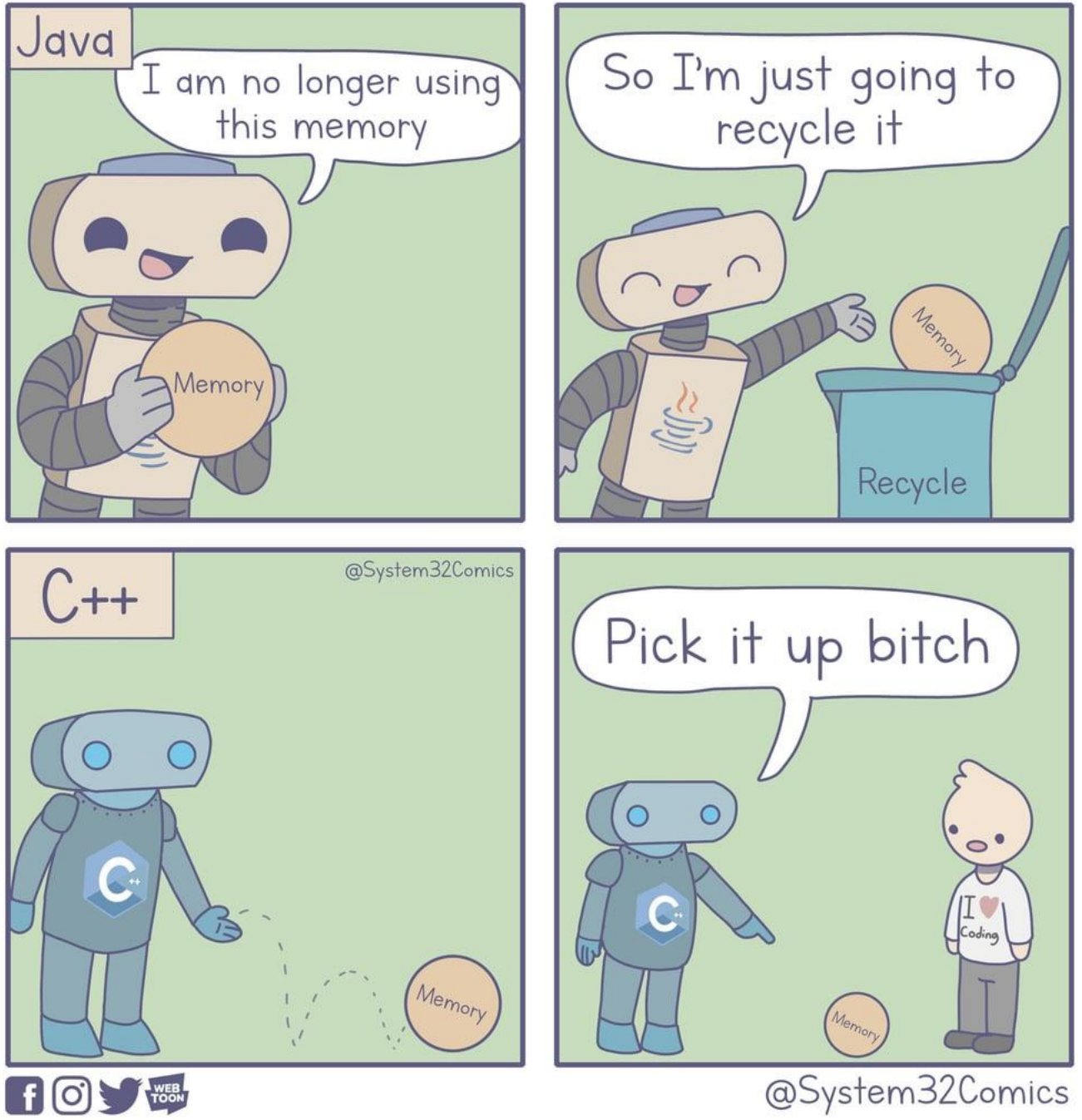
- **High-Level Languages: Abstraction Begins**
  - Introduction of languages like FORTRAN, C, and Pascal.
  - Human-readable syntax (if, while, print).
  - Compiled into machine code for execution.
  - Limited portability across systems.



C++

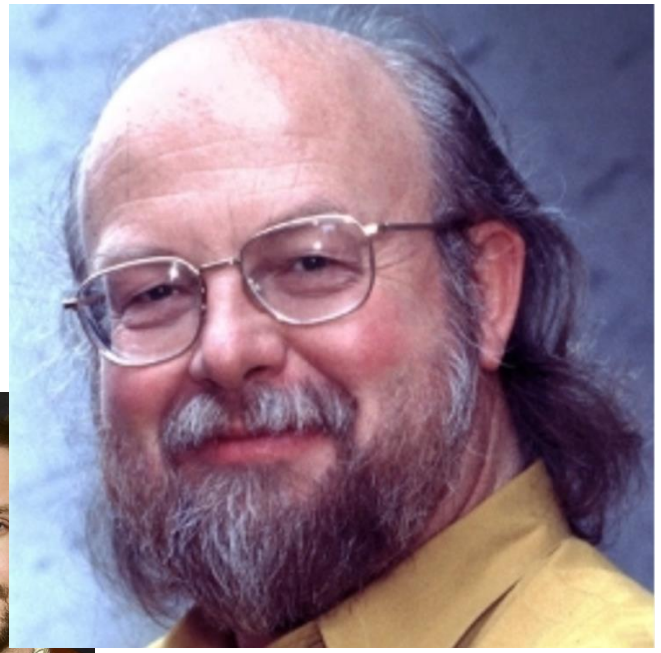
- C++ programs are compiled directly into platform-specific machine code by a compiler (e.g., GCC, MSVC).
- The output is tailored for the operating system and processor architecture where it was compiled.
- C++ often includes platform-specific system calls and libraries, further limiting portability.

- C++ developers manage memory manually, which can lead to undefined behavior on certain platforms if not handled carefully.

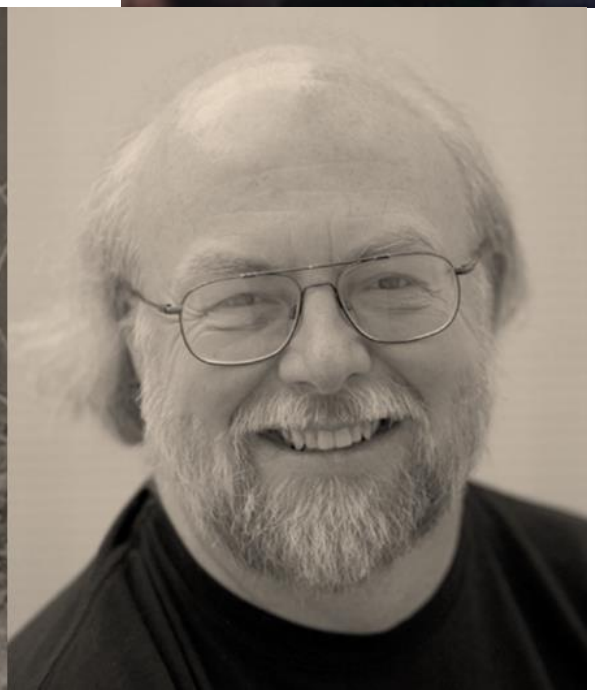


The Birth of Java

- **Introduced by Sun Microsystems in 1995.**
  - Created by James Gosling and his team.
  - Originally designed for interactive television systems.



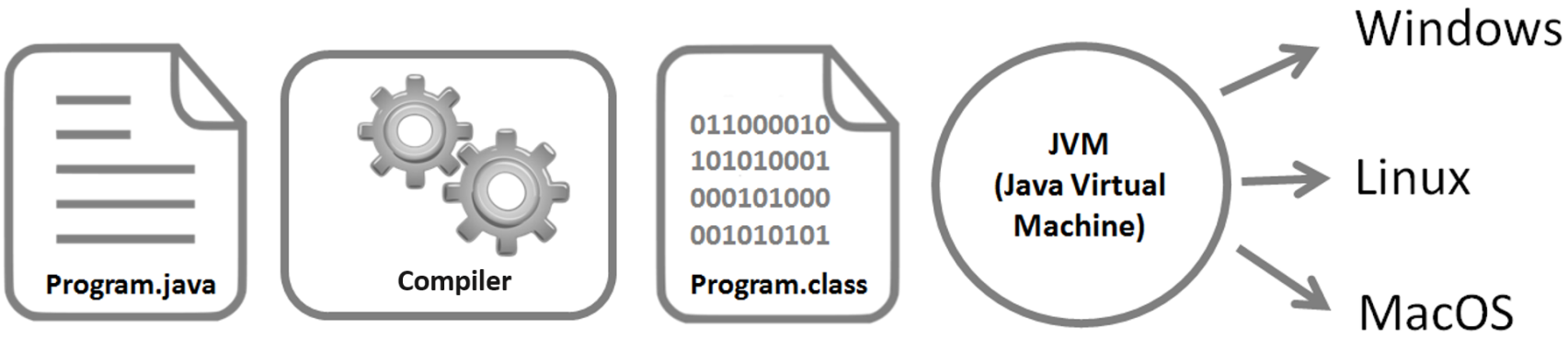

gettyimages
Credit: The AGE



Platform Independence

Write Once, Run Anywhere (WORA)

- Java programs are compiled into **bytecode**, a universal format.
- Bytecode is executed by the JVM (Java Virtual Machine), which is platform-specific.
- This ensures Java programs run on any device with a compatible JVM.
- **Benefit**: Eliminates platform-specific dependencies.



The Mars Rover

Google

eBay

Minecraft

Netflix

Uber

Eclipse IDE

Twitter

Spotify

CashApp

Signal

NASA WorldWind

C++

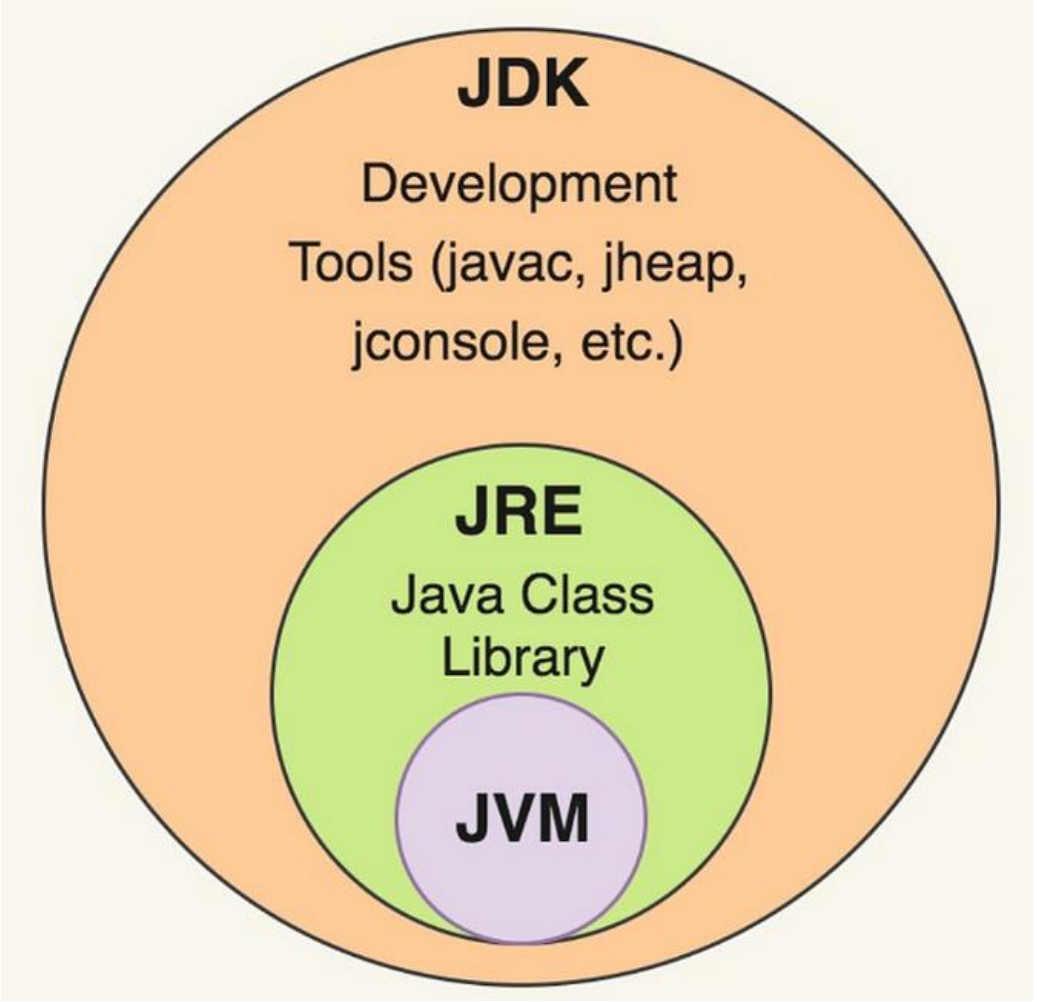
- Source Code (.cpp) -> Compiler -> Assembly Code -> Assembler -> Binary Code (Machine Code)

Java

- Source Code (.java) -> Compiler (javac) -> Bytecode (.class) -> JVM -> Binary Code (Machine Code)

Java Ecosystem Overview

- Key components:
  - **JDK**: Development tools.
  - **JVM**: Runs Java programs.
  - **JRE**: Bridges development and execution.



Java Development Kit (JDK)

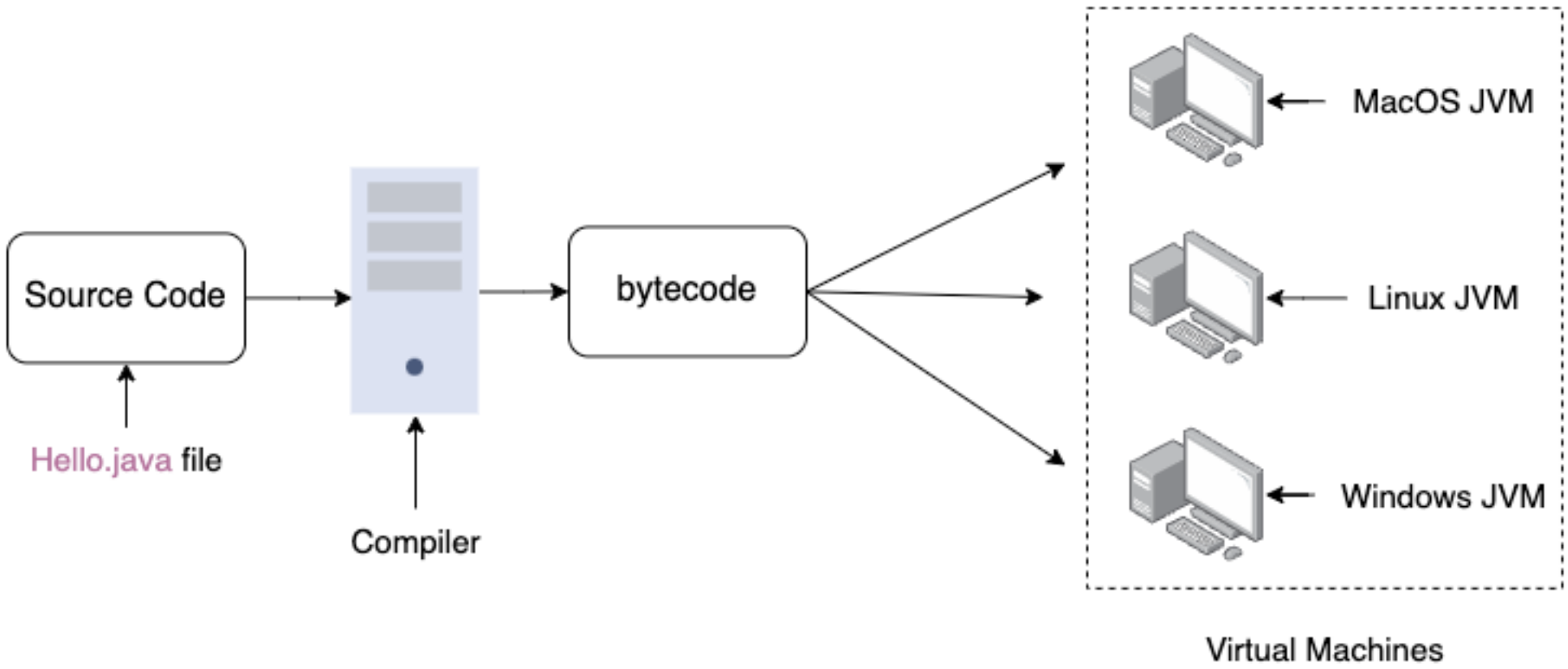
- **Purpose:**
  - A toolkit for developers to write, compile, and debug Java applications.
- **Key Components:**
  - **Compiler (javac)**: Converts source code to bytecode.
  - **Debugger (jdb)**: Helps identify and fix issues.
  - **Libraries and Tools:** Pre-built code and utilities for development.
- **Who uses it?**
  - Developers building Java applications.

Java Virtual Machine (JVM)

- **Purpose:**
  - Runs Java bytecode and ensures platform independence.
- **Responsibilities:**
  - **Bytecode Interpretation:** Converts .class files into machine code.
  - **Memory Management:** Automatic garbage collection.
  - **Security:** Executes code in a sandboxed environment.

- JVM is platform-specific but runs the same bytecode universally.

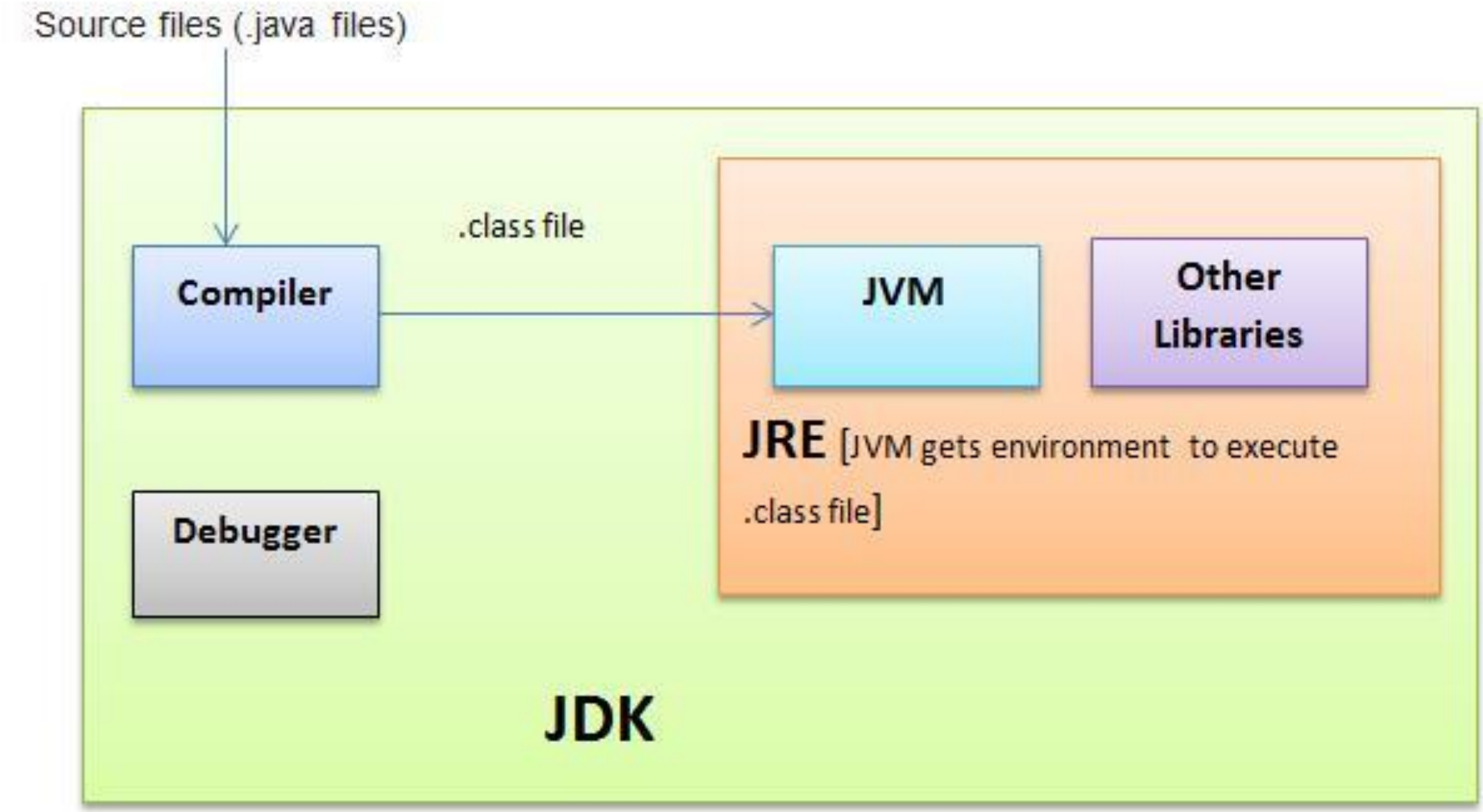
Java Virtual Machine (JVM)



Java Runtime Environment (JRE)

- **Purpose:**
  - Provides everything needed to run Java programs.
- **Components:**
  - **JVM**: Executes bytecode.
  - **Libraries:** Pre-packaged Java APIs for functionality (e.g., java.util, java.io).
  - **Other Tools:** Assists program execution (e.g., java command).

- **Who uses it?** End-users running Java applications, not writing them.



Source Code → JDK (javac) → Bytecode (.class) → JRE (JVM) → Binary Code.

Java basics

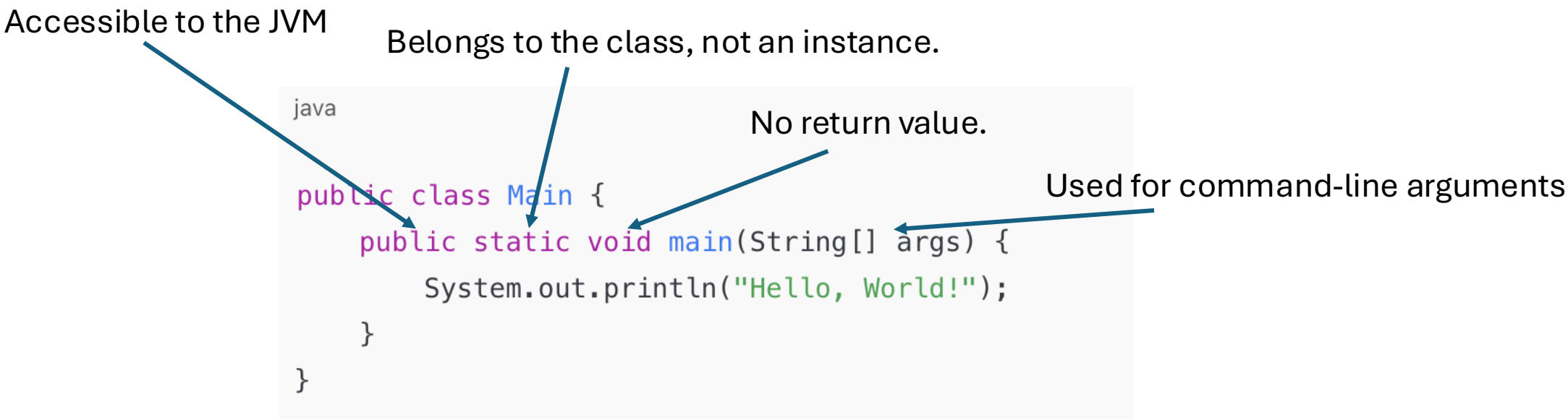
Strong Typing

- **Strong Typing:**
  - Variables must be declared with a specific type (int, String, etc.).
  - Prevents accidental type errors during development.
- **Reliability:**
  - Robust exception handling ensures stability.
  - *Garbage collection* manages memory automatically.
  - Security features prevent unauthorized operations.

- Java's design focuses on consistent, error-free execution.

The Main Class

- Every Java program starts execution from the **main** method
- The entry point of a Java program
- The JVM looks for the main method to begin execution.



Java Output

- *System.out.println:* Used to print messages to the console.

```
System.out.println("Hello, Java!"); // Prints with a newline
System.out.print("No newline");    // Prints without a newline
System.out.printf("Formatted number: %.2f", 3.14159); // Formatted output
```

- **What is System.out?**
  - A built-in Java class in the java.lang package.
  - Provides access to system-related functionality.

What is ”*static*”

- Java requires methods to exist inside classes.
- The *static* keyword allows calling methods without creating an object

```
public class MathUtils {
    public static int add(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        System.out.println(add(5, 3)); // Output: 8
    }
}
```

Java Input

- Use the *Scanner* class for input

- **Key Methods:**
- nextLine(): Reads a line of text
- nextInt(), nextDouble():
  - Reads numbers

```java
import java.util.Scanner;

public class InputExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = scanner.nextLine();
        System.out.println("Hello, " + name + "!");
    }
}
```

Java Data Types

Java supports two main types:

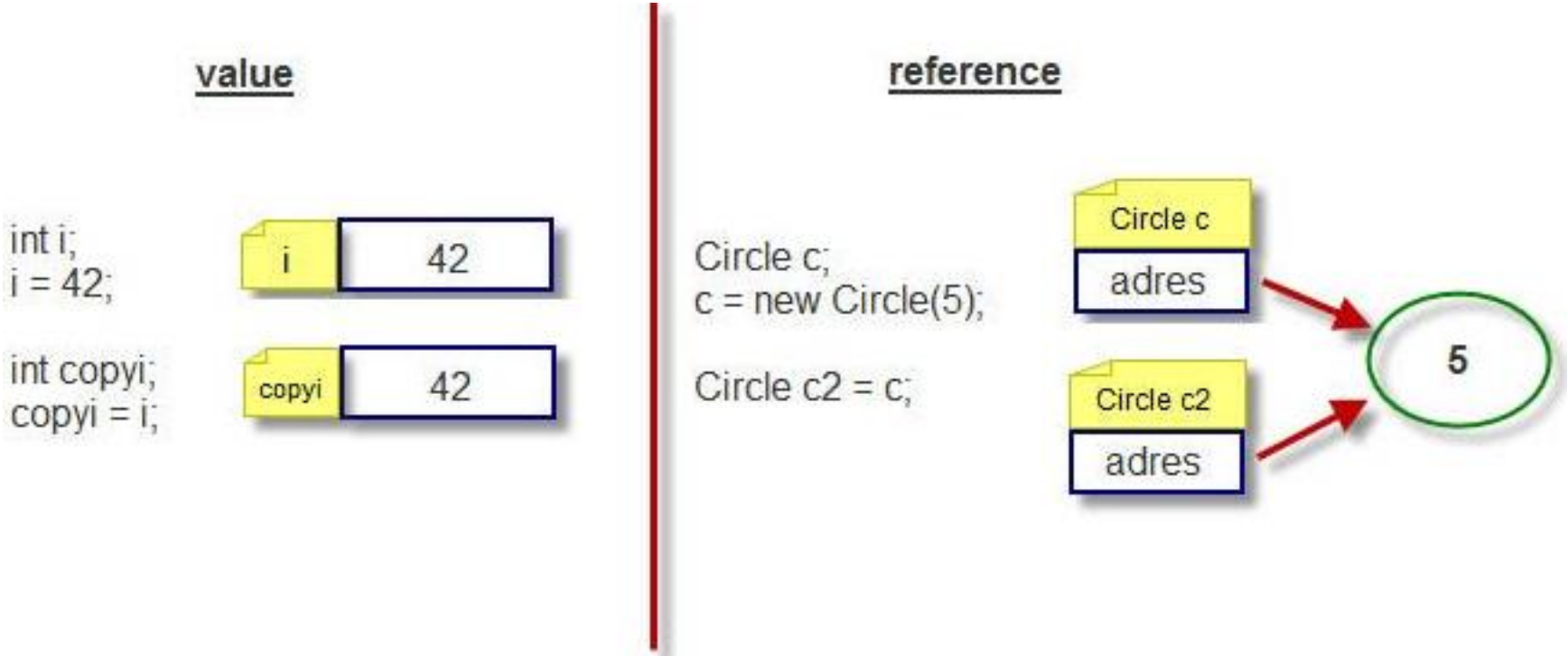
- **Primitive Types:**
  - byte, short, int, long (integer types).
  - float, double (floating-point numbers).
  - char (single characters).
  - boolean (true/false).
- **Reference Types:**
  - Objects and arrays.

```java
int number = 42;
double pi = 3.14;
boolean isJavaFun = true;
String message = "Hello, Java!";
```

- **Why use them?**
  - Primitives: For efficiency and speed.
  - Reference types: For flexibility and object-oriented programming.

Reference type

- Instead of holding the actual value, a reference type **stores a reference (or address)** that points to the memory location where the object or data is stored.

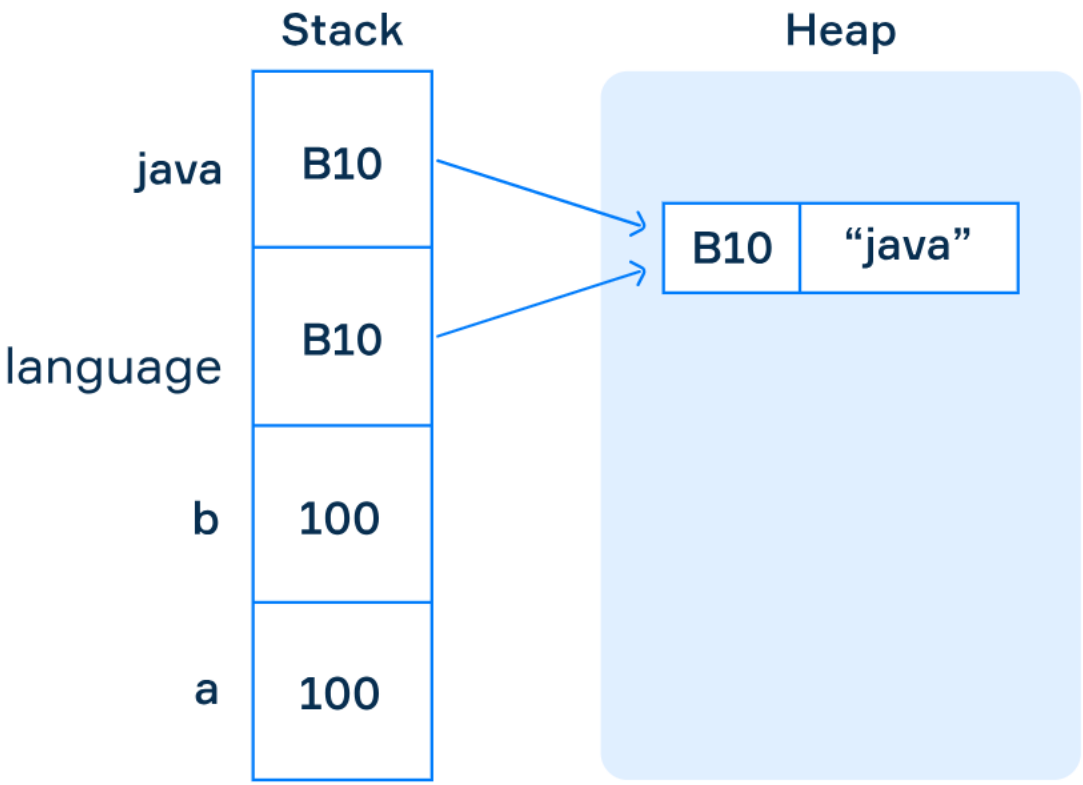


Stack and Heap

- Reference types use **heap memory** for dynamic allocation of objects.
- The reference itself resides in the stack memory.

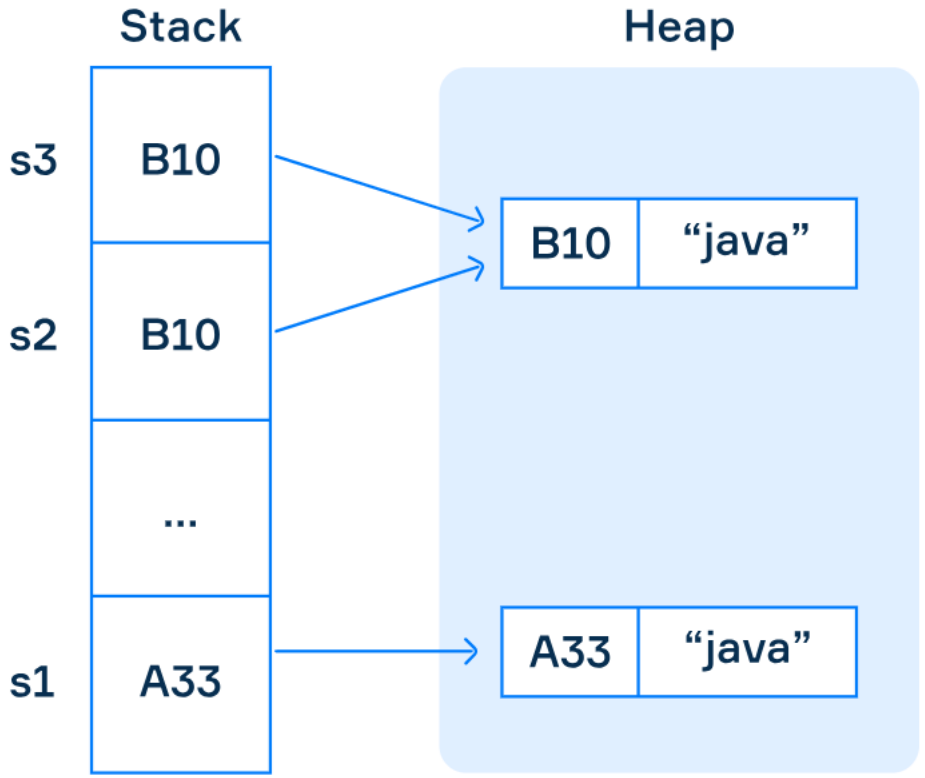
- **stack** and **heap** are types of memory areas used by the Java Virtual Machine (JVM)

```java
int a = 100;
int b = a; // 100 is copied to b

String language = new String("java");
String java = language;
```



Stack and Heap

```java
String s1 = new String("java");
String s2 = new String("java");
String s3 = s2;

System.out.println(s1 == s2); // false
System.out.println(s2 == s3); // true
```



- **Stack:** Temporary storage for method execution, local variables, and references.
- **Heap:** Long-term storage for objects and shared data, managed by the garbage collector.

TYPE	DESCRIPTION	DEFAULT	SIZE	EXAMPLE LITERALS	RANGE OF VALUES
boolean	true or false	false	1 bit	true, false	true, false
byte	twos complement integer	0	8 bits	(none)	-128 to 127
char	unicode character	\u0000	16 bits	'a', '\u0041', '\101', '\\', '\'','\n',' β'	character representation of ASCII values 0 to 255
short	twos complement integer	0	16 bits	(none)	-32,768 to 32,767
int	twos complement integer	0	32 bits	-2, -1, 0, 1, 2	-2,147,483,648 to 2,147,483,647
long	twos complement integer	0	64 bits	-2L, -1L, 0L, 1L, 2L	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	IEEE 754 floating point	0.0	32 bits	1.23e100f, -1.23e-100f, .3f, 3.14F	upto 7 decimal digits
double	IEEE 754 floating point	0.0	64 bits	1.23456e300d, -1.23456e-300d, 1e1d	upto 16 decimal digits

Wrappers

- Wrapper classes provide an object representation for primitive types.
- Collections like ArrayList can only store objects.

```
Integer num = 10;
System.out.println(num.toString());   // "10"
```

- Autoboxing / Unboxing
  - Conversion of a wrapper type to its primitive and back
  - It is automatic

Primitive Type	Wrapper Class
byte	Byte
boolean	Boolean
char	Character
double	Double
float	Float
int	Integer
long	Long
short	Short

What is the *new* Keyword in Java?

- The *new* keyword is used to **create objects** in Java.
- It dynamically allocates memory in the **heap** for an object.

```java
ClassName objectName = new ClassName();
```

- Instantiates objects from classes.
- Returns a reference to the created object.

Steps when new is used

1. Allocates memory for the object in the heap.
2. Initializes instance variables with default values.
3. Invokes the **constructor** to set initial values.
4. Returns the memory reference to the object.

```
public class Car {
    String model;
    int year;

    void drive() {
        System.out.println("Driving...");
    }
}
```

```
Car myCar = new Car();
myCar.model = "Tesla";
myCar.year = 2023;
myCar.drive();
```

Constructors in Java

What is a Constructor?

- A special method used to initialize objects.
- Same name as the class.
- No return type.

```java
public class Car {
    String model;
    int year;

    // Constructor
    public Car(String model, int year) {
        this.model = model;
        this.year = year;
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        Car myCar = new Car("Tesla", 2023);
        System.out.println(myCar.model + " - " + myCar.year);
    }
}
```

Garbage Collector

What are Destructors?

- In languages like C++, a destructor is a special method called automatically when an object is destroyed or goes out of scope.
  - Release resources like memory, file handles, or network connections
  - Prevent resource leaks

- Java **does not have** destructors because
  - Java's Garbage Collector automatically reclaims memory for unused objects
  - When there are no more references to an object, it becomes eligible for garbage collection
  - there’s no guarantee when an object will be garbage collected

Common Tools for Java Development

Key Categories:

- **IDEs** for writing and debugging code.
- **Build Tools** for automating compilation and packaging.
- **Package Management** for managing libraries and dependencies.

Integrated Development Environments (IDEs)

- Tools that provide an integrated workspace for coding, debugging, and testing Java applications.
- **Popular IDEs:**
  - **IntelliJ IDEA**
  - **Eclipse**
  - **NetBeans**

- Code editor with syntax highlighting and autocompletion
- Debugger for stepping through code
- Integrated build and version control tools
- Support for plugins

Build Tools

- Automate tasks like compiling code, packaging applications, and managing dependencies.
- **Popular Build Tools:**
  - **Maven:**
    - Uses an XML file (pom.xml) for project configuration.
    - Focuses on dependency management and standard project structures.
  - **Gradle:**
    - Uses a Groovy or Kotlin-based build script.

CSC1031: OOP

Week 2 cont.

Why Should the Filename Match the Class Name in Java?

1. In Java, the filename **must** match the name of the **public class** in the file.
2. Example:
    1. If the class is named Cake, the file should be Cake.java.
3. The Java compiler relies on this rule to **locate and compile** your code correctly.
4. A file can contain multiple classes, but **only one class can be public**, and that class determines the filename.
5. Matching filenames and class names helps maintain **organization** in larger projects.

6.Violation Consequences:

1. Mismatched names lead to a **compilation error**:

Error: Class Cake is public, should be declared in a file named Cake.java

Is it possible to have multiple classes in one file?

- Yes, but only one class in the file can be declared public.
- The filename still has to match the public class name.
- Other non-public classes in the file can be named differently
- but they won't be accessible outside their *package* unless explicitly referenced

```java
// File: HelloWorld.java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}


class Helper {
    void assist() {
        System.out.println("I'm here to help!");
    }
}
```

Class Components

- **Attributes (Fields):**
  - These represent the **state or properties**
  - typically declared as **private** to ensure **encapsulation**
- **Constructor:**
  - A special **method** used to initialize objects of the class
  - has the **same name as the class** and does **not have a return type**
- **Methods:**
  - Define the **behavior** of the class or the actions it can perform
  - Methods can:
    - Access or modify the class's attributes.
    - Perform specific tasks.
- **Other Components:**
  - **Inner Classes:**
    - Classes defined within other classes for grouping purposes.
  - **Static Fields/Methods:**
    - Shared across all objects of the class.

```
1  class ClassName {
2      // Attributes (Fields)
3      private DataType attributeName;
4
5      // Constructor
6      public ClassName() {
7          // Initialize attributes
8      }
9
10     // Methods (Behaviors)
11     public ReturnType methodName() {
12         // Code defining the behavior
13     }
14
15     // Additional Methods, Inner Classes, etc.
16 }
```

Attributes vs. Local Variables:

- **Attributes**:
  - Belong to the class (defined outside methods)
- **Local Variables**:
  - Exist only **inside methods** and are not accessible outside the method's scope.

```java
class Example {
    private int globalValue; // Attribute (class-level)

    void showValue() {
        int localValue = 10; // Local variable
        System.out.println("Global: " + globalValue);
        System.out.println("Local: " + localValue);
    }
}
```

What is *this*?

- A special **reference keyword** in Java
- Refers to the **current object** (the instance of the class in which it is used).

```
4  class Main {
5      int a;
6      int b;
7
8      public void setData(int a, int b) {
9          a = a;
10         b = b;
11     }
12 }
```

```
4  class Main {
5      int a;
6      int b;
7
8      public void setData(int new_a, int new_b) {
9          a = new_a;
10         b = new_b;
11     }
12 }
```

```
4  class Main {
5      int a;
6      int b;
7
8      public void setData(int a, int b) {
9          this.a = a;
10         this.b = b;
11     }
12 }
```

When to use this?

- Passing the Current Object as an Argument:

Methods needs a Person object

```
1  class Person {
2      private String name;
3      ...
4
5      void display(Person obj) {
6          System.out.println("Name: " + obj.name);
7      }
8
9      void show() {
10         display(this);  // Pass current object
11     }
12 }
```

When You Don’t Need *this*?

- No Name Conflicts
- No Ambiguity
- Static Context

```java
class Person {
    private String name;

    public void display() {
        System.out.println(name);
        // Direct access, no 'this' needed
    }

    public void setName(String newName) {
        name = newName;
        // No conflict, no 'this' needed
    }
}
```

```java
class Person {
    private String name;

    public static void staticMethod() {
        // this.name = "Alice";
        // Error: Cannot use 'this' in static context
    }
}
```

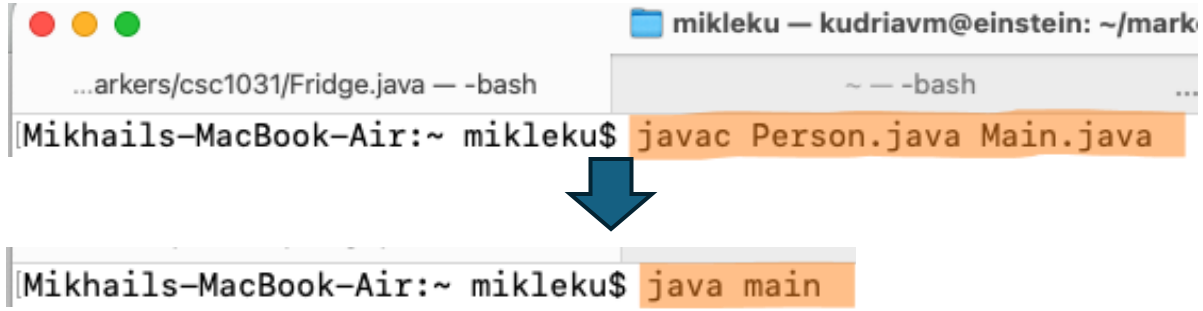
How to Test Your Own Class in Java

Write a ”runner”

```java
class Person {
    private String name;
    private int age;

    ...
}
```

```java
public class Main {
    public static void main(String[] args) {
        // Create an object of the Person class
        Person person = new Person("Alice", 25);

        // Test the constructor and getters
        System.out.println("Name: " + person.getName());
        System.out.println("Age: " + person.getAge());

        // Test setters
        person.setName("Bob");
        person.setAge(30);

        System.out.println(person.getName());
        System.out.println(person.getAge());
    }
}
```

```
.
└── your_folder/
    ├── Person.java
    └── Main.java
```



How to Test Your Own Class in Java

```java
class Person {
    private String name;
    private int age;

    ...
}
```

Write a ”runner”

```java
public class Main {
    public static void main(String[] args) {
        // Create an object of the Person class
        Person person = new Person("Alice", 25);

        // Test the constructor and getters
        System.out.println("Name: " + person.getName());
        System.out.println("Age: " + person.getAge());

        // Test setters
        person.setName("Bob");
        person.setAge(30);

        System.out.println(person.getName());
        System.out.println(person.getAge());
    }
}
```

Test function within the same file

```java
class Person {
    private String name;
    private int age;

    ...
    public static void main(String[] args) {
        // Create an object of the Person class
        Person person = new Person("Alice", 25);

        // Test the constructor and getters
        System.out.println("Name: " + person.getName());
        System.out.println("Age: " + person.getAge());


    }
}
```

Eratosthenes sieve

- The **sieve** of **Eratosthenes** is an ancient **algorithm** for finding all prime numbers up to any given limit

```
algorithm Sieve of Eratosthenes is
    input: an integer n > 1.
    output: all prime numbers from 2 through n.

    let A be an array of Boolean values, indexed by integers 2 to n,
    initially all set to true.

    for i = 2, 3, 4, ..., not exceeding √n do
        if A[i] is true
            for j = i², i²+i, i²+2i, i²+3i, ..., not exceeding n do
                A[j] := false

    return all i such that A[i] is true.
```

Proper data types

- Array
  - [0..100.000.000]
- Boolean:
  - 10^8 bit
  - 1250000 Byte
  - ~12 MByte

- int
  - 32*10^8 bit
  - 4*10^8 byte
  - 380 MByte

Type	Size (in bits)	Range
byte	8	-128 to 127
short	16	-32,768 to 32,767
int	32	-2^{31} to $2^{31}-1$
long	64	-2^{63} to $2^{63}-1$
float	32	1.4e-045 to 3.4e+038
double	64	4.9e-324 to 1.8e+308
char	16	0 to 65,535
boolean	1	true or false

Constructors in Java

What is a Constructor?

- A special method used to initialize objects.
- Same name as the class.
- No return type.

```java
public class Car {
    String model;
    int year;

    // Constructor
    public Car(String model, int year) {
        this.model = model;
        this.year = year;
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        Car myCar = new Car("Tesla", 2023);
        System.out.println(myCar.model + " - " + myCar.year);
    }
}
```

Garbage Collector

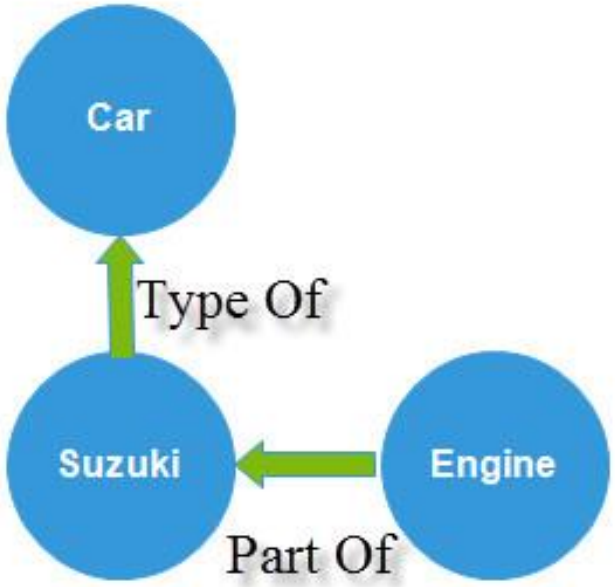
What are Destructors?

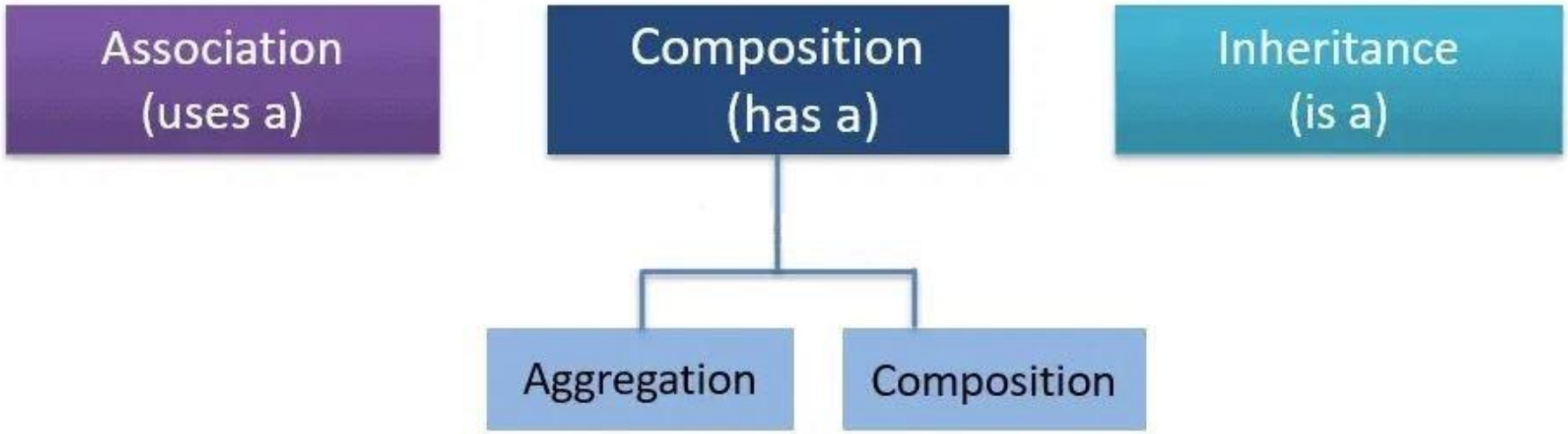
- In languages like C++, a destructor is a special method called when an object is destroyed or goes out of scope.
  - Release resources like memory, file handles, or network connections
  - Prevent resource leaks

- Java **does not have** destructors because
  - Java's Garbage Collector automatically reclaims memory for unused objects
  - When there are no more references to an object, it becomes eligible for garbage collection
  - there’s no guarantee when an object will be garbage collected

Understanding Object Relationships

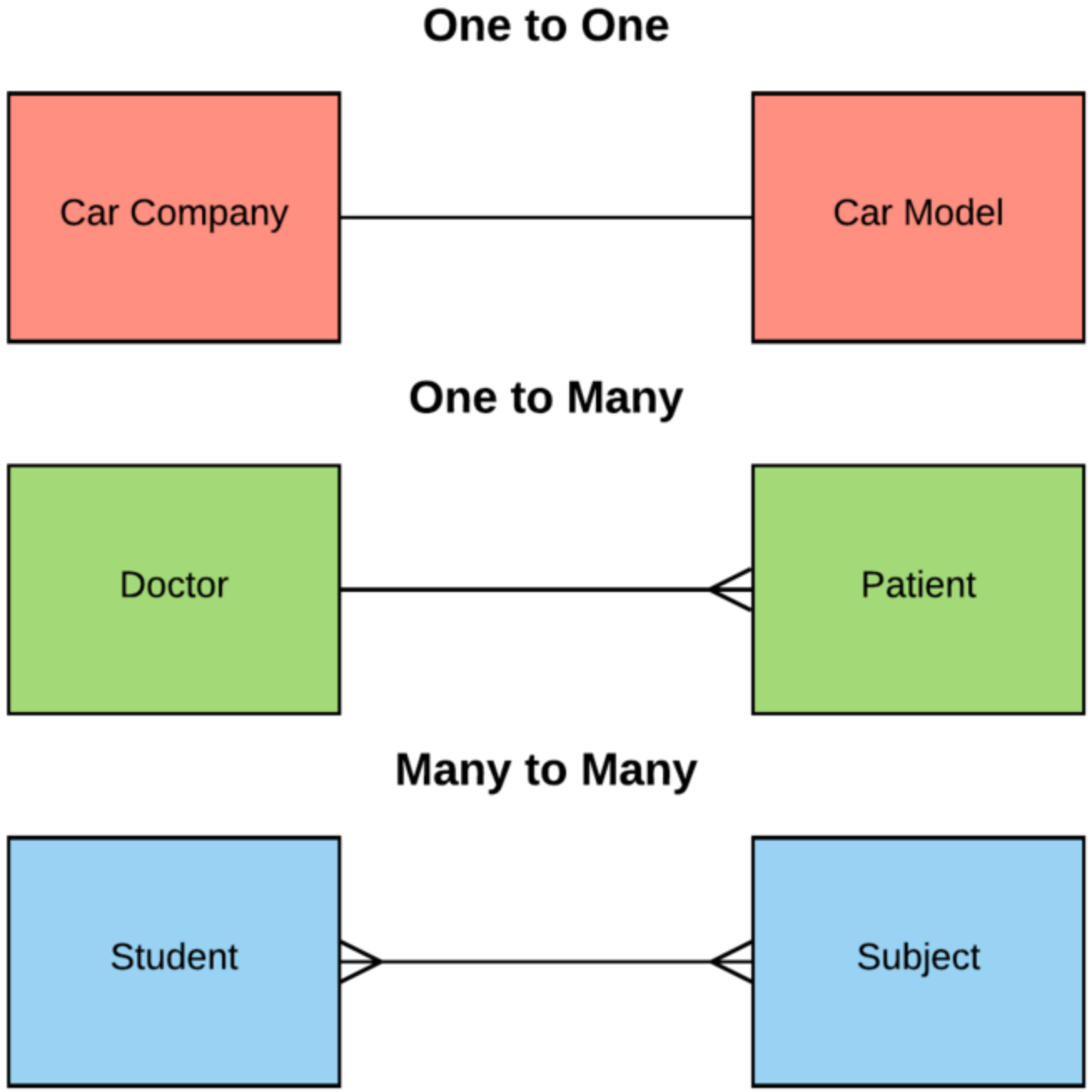
- **Association**
  - one object uses another
- **Aggregation**
  - "Has-a"
- **Composition**
  - Strong "Has-a"





Association

- A **generic relationship** between two classes, where one class uses or interacts with another
- one class "knows about" or "uses" another class
- no ownership involved. Both classes exist independently.
- Types:
  - One-to-One
  - One-to-Many
  - Many-to-Many

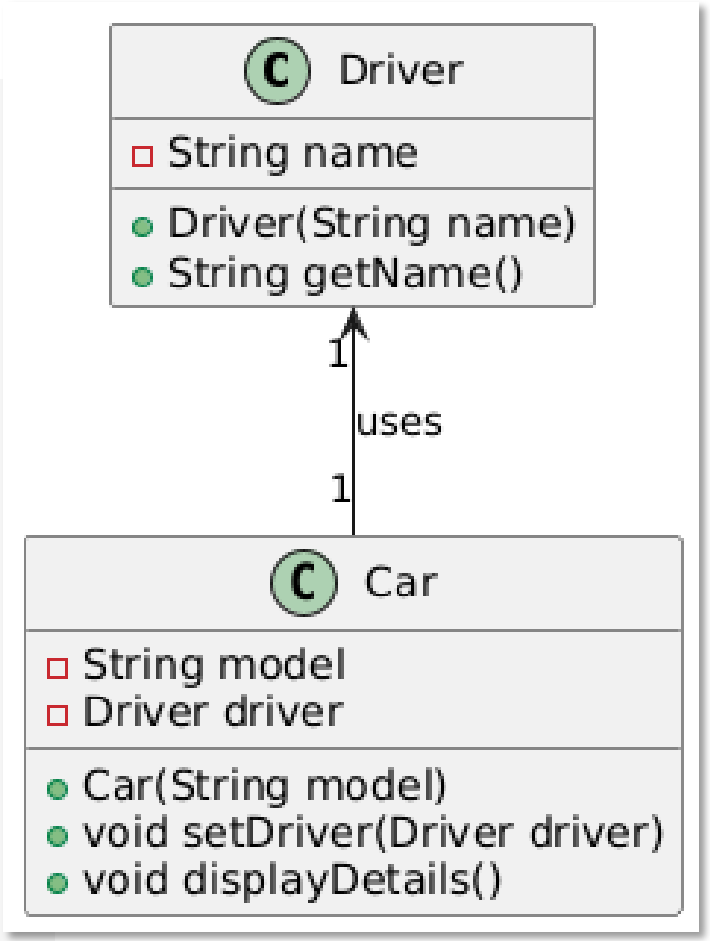


Association

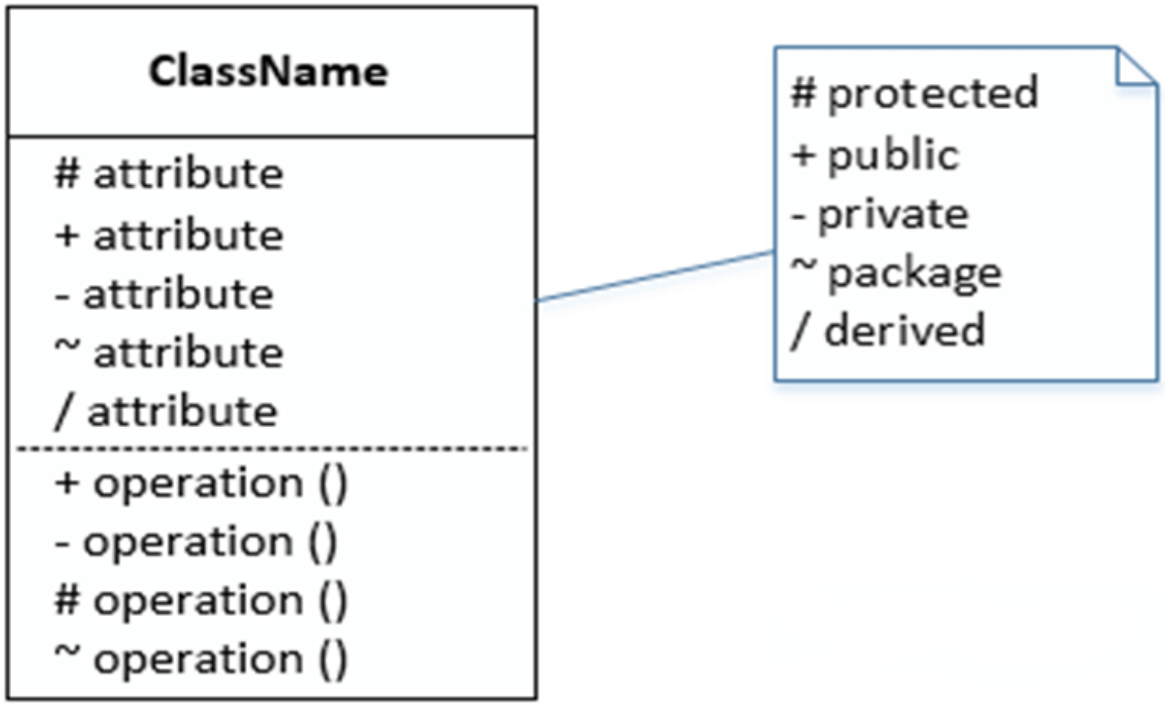
```
1  class Driver {
2      private String name;
3      public Driver(String name) { this.name = name; }
4      public String getName() { return name; }
5  }
6
```

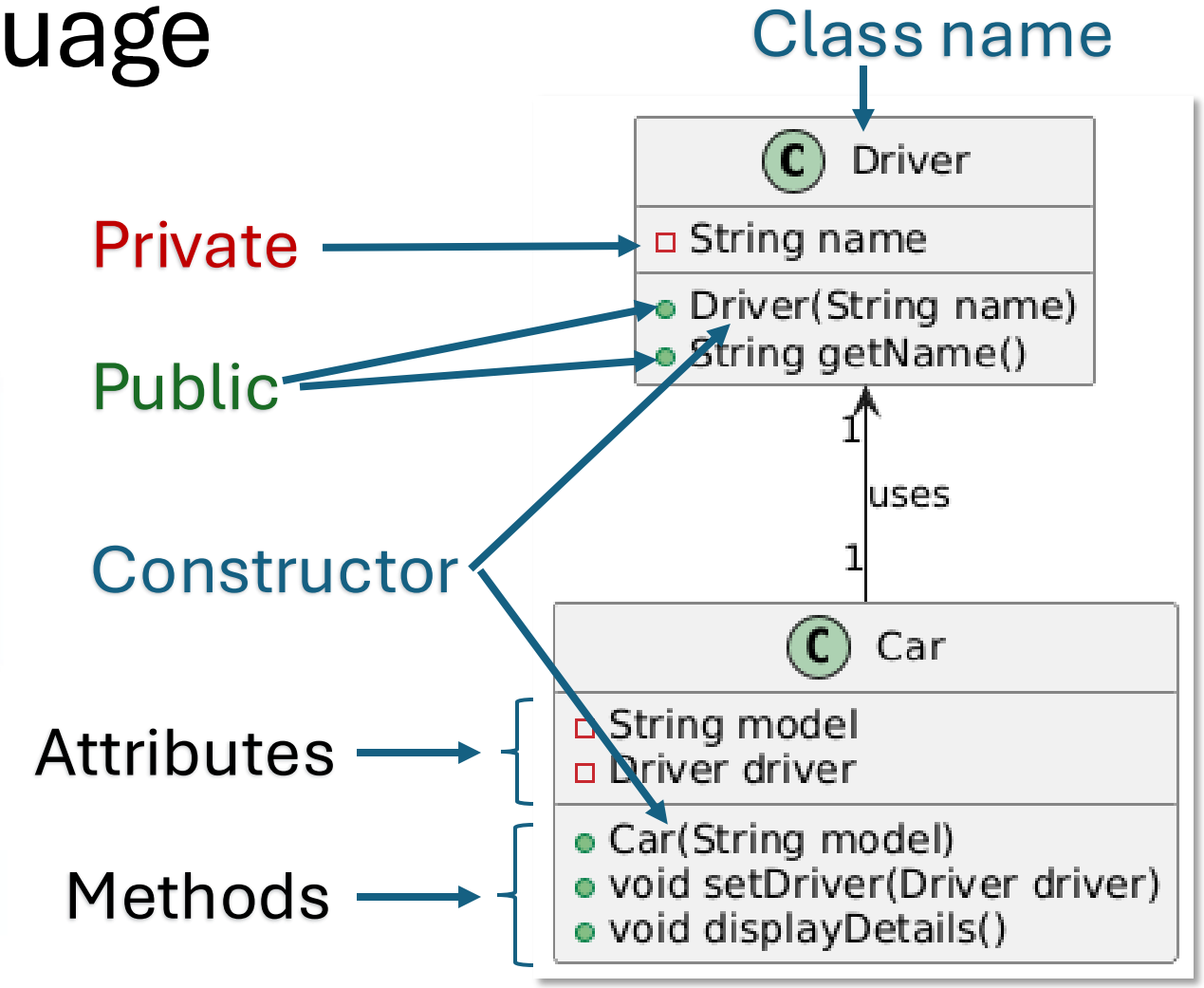
Association

```java
class Driver {
    private String name;
    public Driver(String name) { this.name = name; }
    public String getName() { return name; }
}

class Car {
    private String model;
    private Driver driver;

    public Car(String model) { this.model = model; }
    public void setDriver(Driver driver) { this.driver = driver; }
    public void displayDetails() {
        System.out.println("Car: " + model);
        System.out.println("Driver: " + driver.getName());
    }
}
```



Unified Modeling Language UML





Aggregation

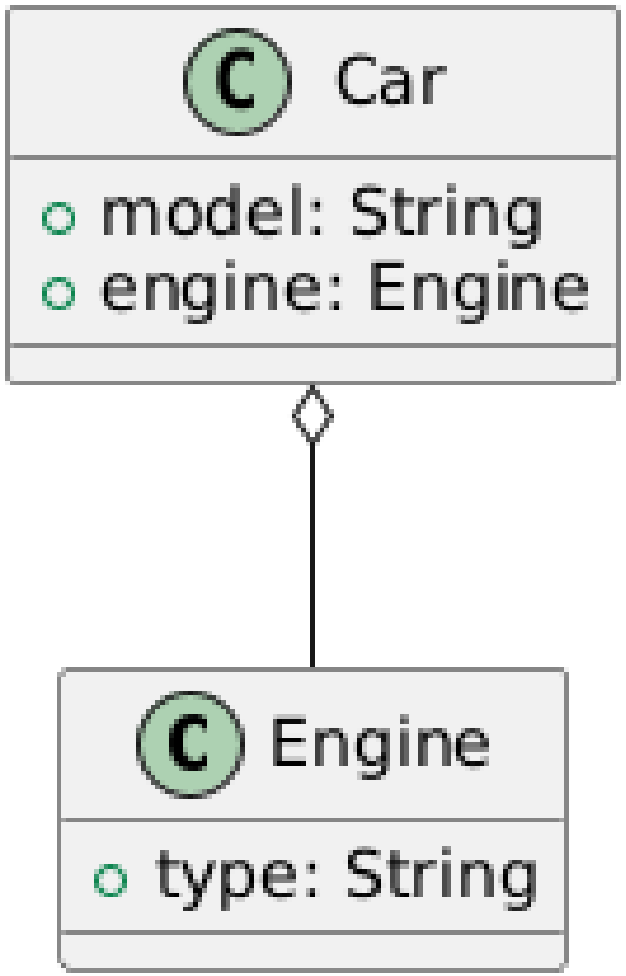
- A **"has-a" relationship** where one class contains or references another
- The contained object can exist independently of the container
- It represents a **weak ownership**

Aggregation

```
7  class Car {
8      private String model;
9      private Engine engine;
10
11     public Car(String model, Engine engine) {
12         this.model = model;
13         this.engine = engine;
14     }
15     public void displayDetails() {
16         System.out.println("Car: " + model);
17         System.out.println("Engine: " + engine.getType());
18     }
19 }
```

Aggregation

```java
class Engine {
    private String type;
    public Engine(String type) { this.type = type; }
    public String getType() { return type; }
}

class Car {
    private String model;
    private Engine engine;

    public Car(String model, Engine engine) {
        this.model = model;
        this.engine = engine;
    }
    public void displayDetails() {
        System.out.println("Car: " + model);
        System.out.println("Engine: " + engine.getType());
    }
}
```

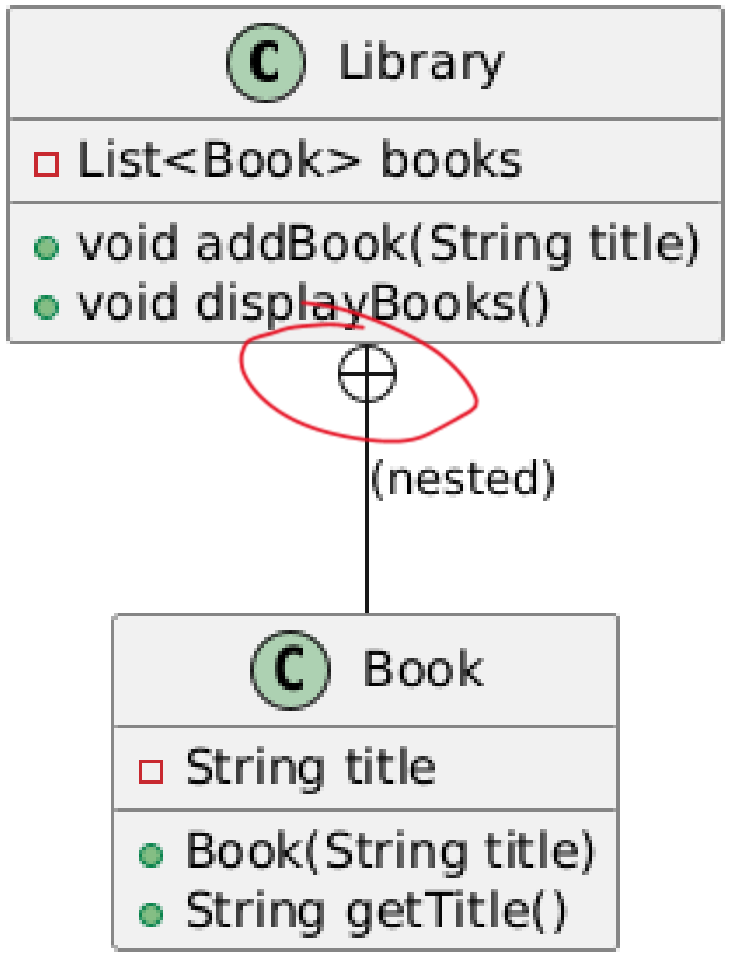


Composition

- A **strong "has-a" relationship** where one class contains or owns another,
- The contained object **cannot exist independently** of the container.
- If the container is destroyed, the contained object is also destroyed.

Composition

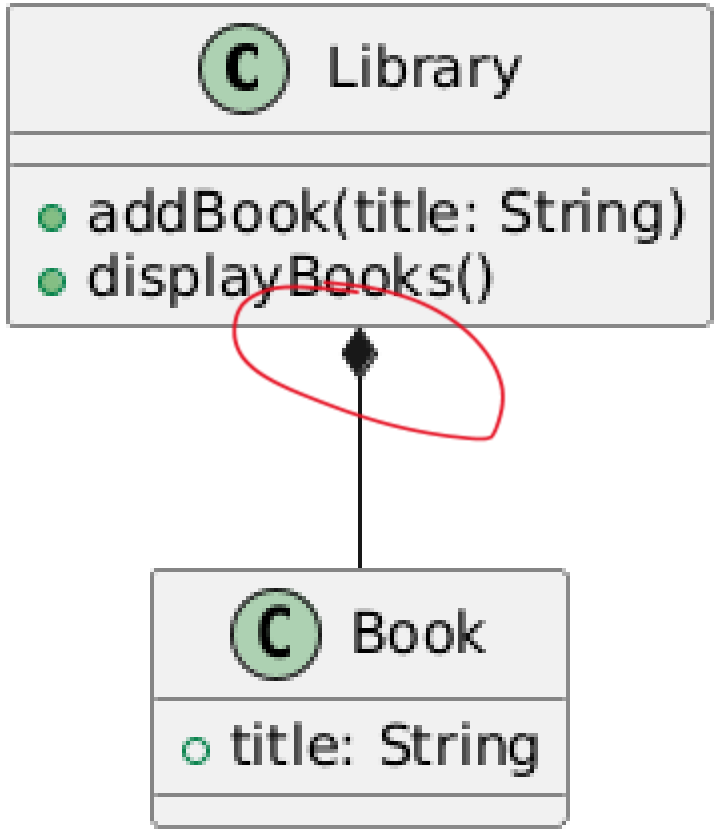
```java
class Library {
    private List<Book> books = new ArrayList<>();

    public void addBook(String title) {
        books.add(new Book(title));
    }

    public void displayBooks() {
        for (Book book : books) {
            System.out.println("Book: " + book.getTitle());
        }
    }

    private class Book {
        private String title;
        public Book(String title) { this.title = title; }
        public String getTitle() { return title; }
    }
}
```



Composition

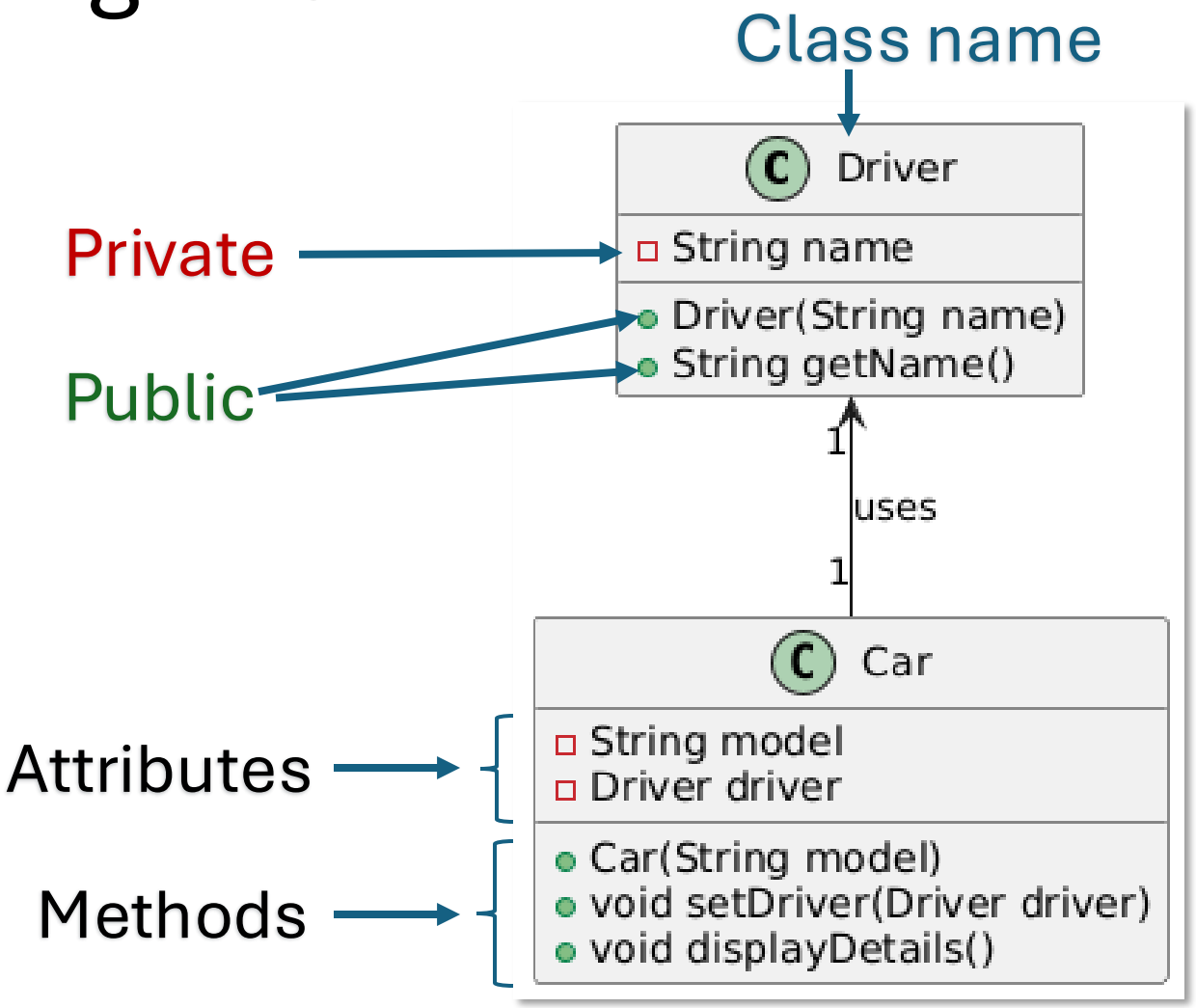
The only place where Book objects exist

```
4  class Library {
5      private List<Book> books = new ArrayList<>();
6
7      public void addBook(String title) {
8          books.add(new Book(title));
9      }
10
11     public void displayBooks() {
12         for (Book book : books) {
13             System.out.println("Book: " + book.getTitle());
14         }
15     }
16 }
17
18 class Book {
19         private String title;
20         public Book(String title) { this.title = title; }
21         public String getTitle() { return title;  }
22 }
```

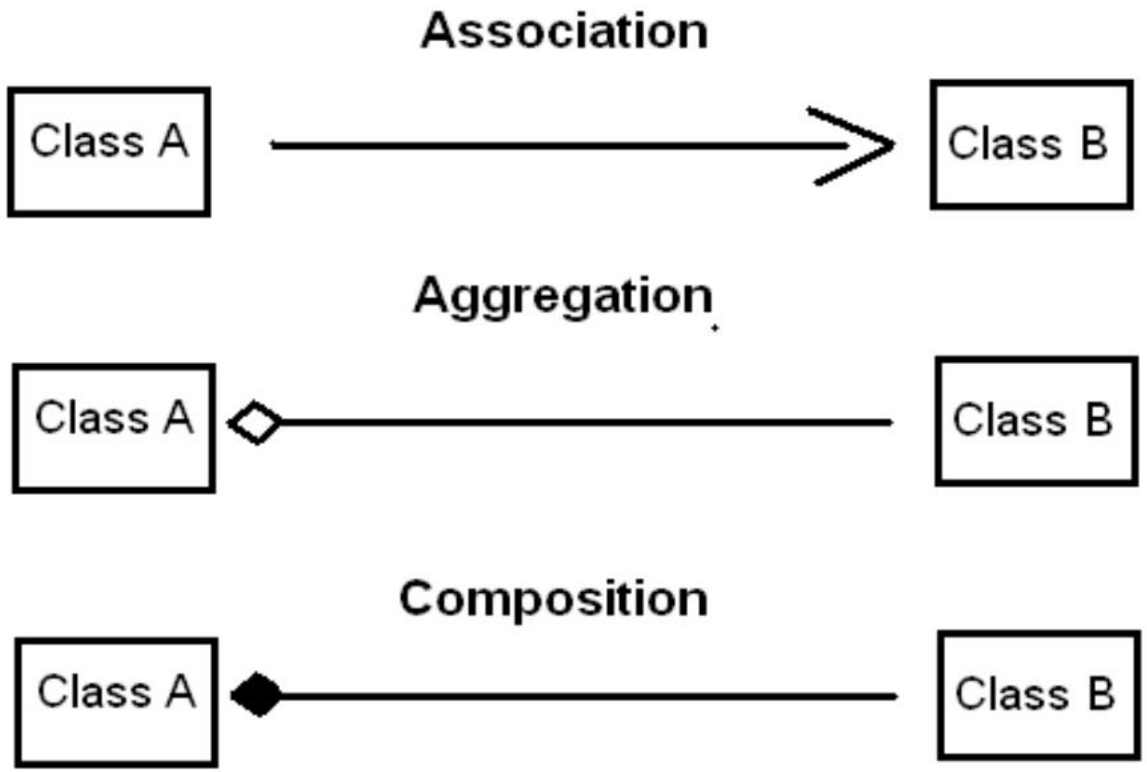


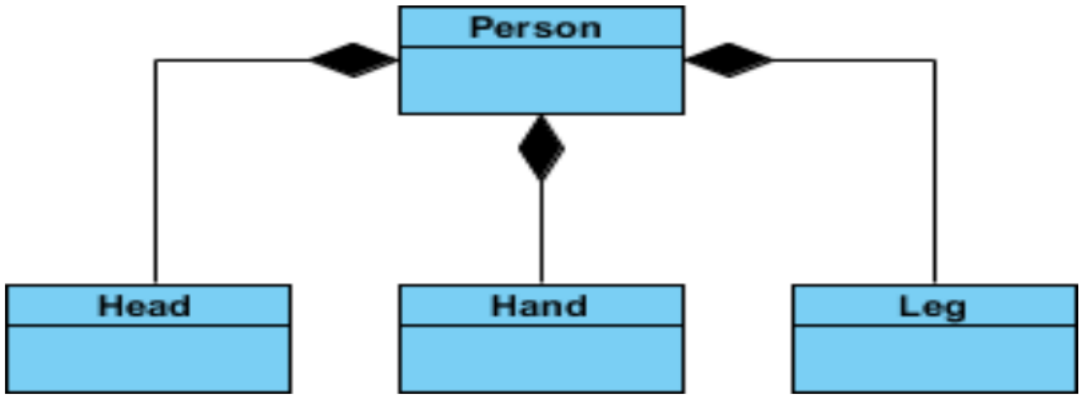
Unified Modeling Language - UML

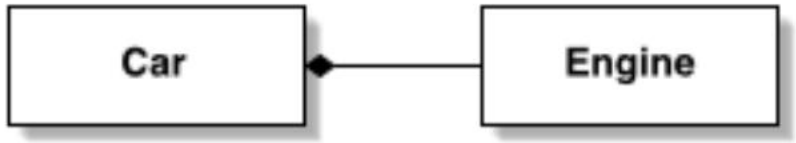
- Classes:
  - Represented as boxes with three sections (Name, Attributes, Methods).
- Relationships: Indicate how classes interact.
  - **Association:** Simple arrow.
  - **Aggregation:** Hollow diamond.
  - **Composition:** Filled diamond.



UML







Composition: every car has an engine.

Aggregation: cars may have passengers, they come and go

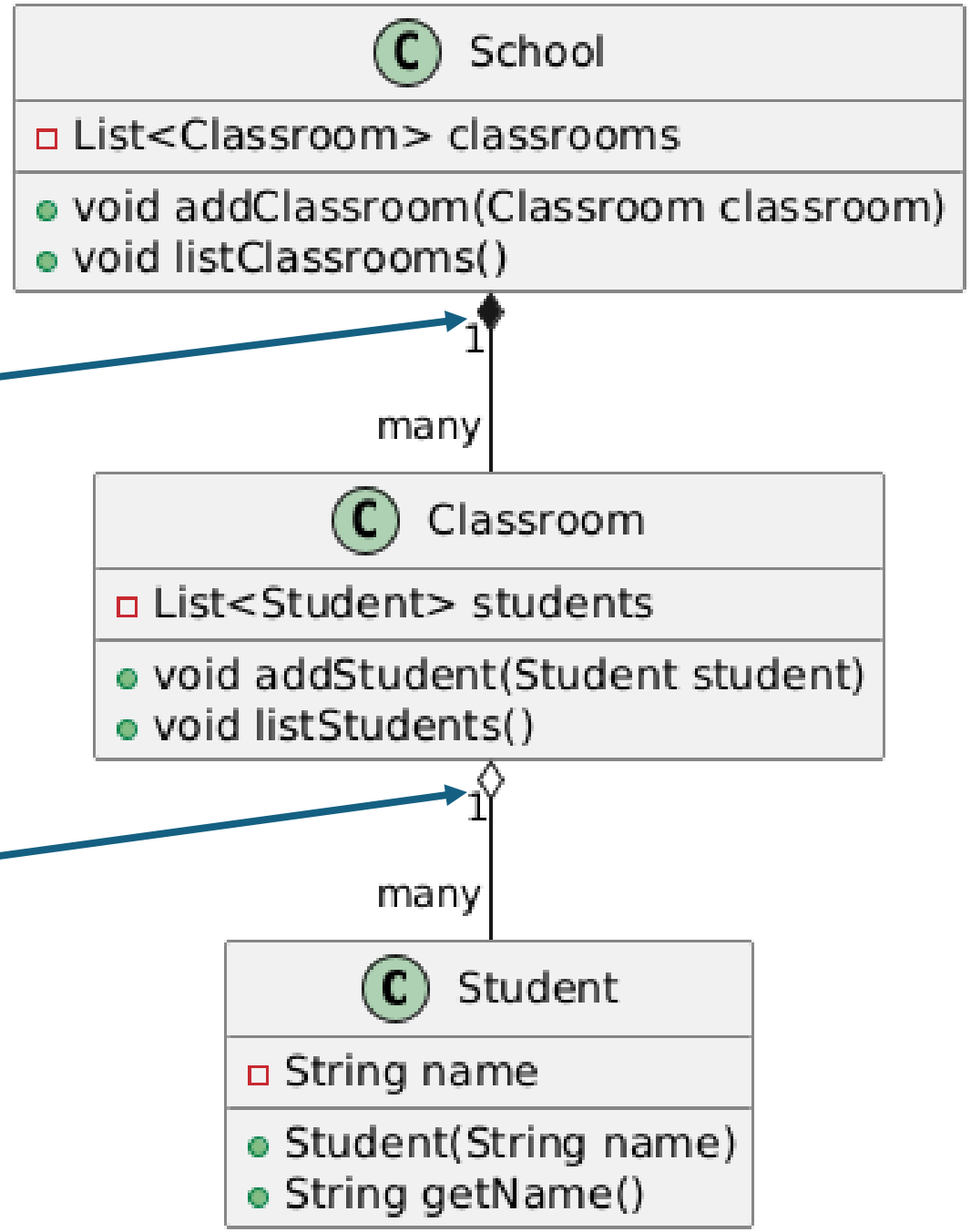
Classrooms can’t exist without a school

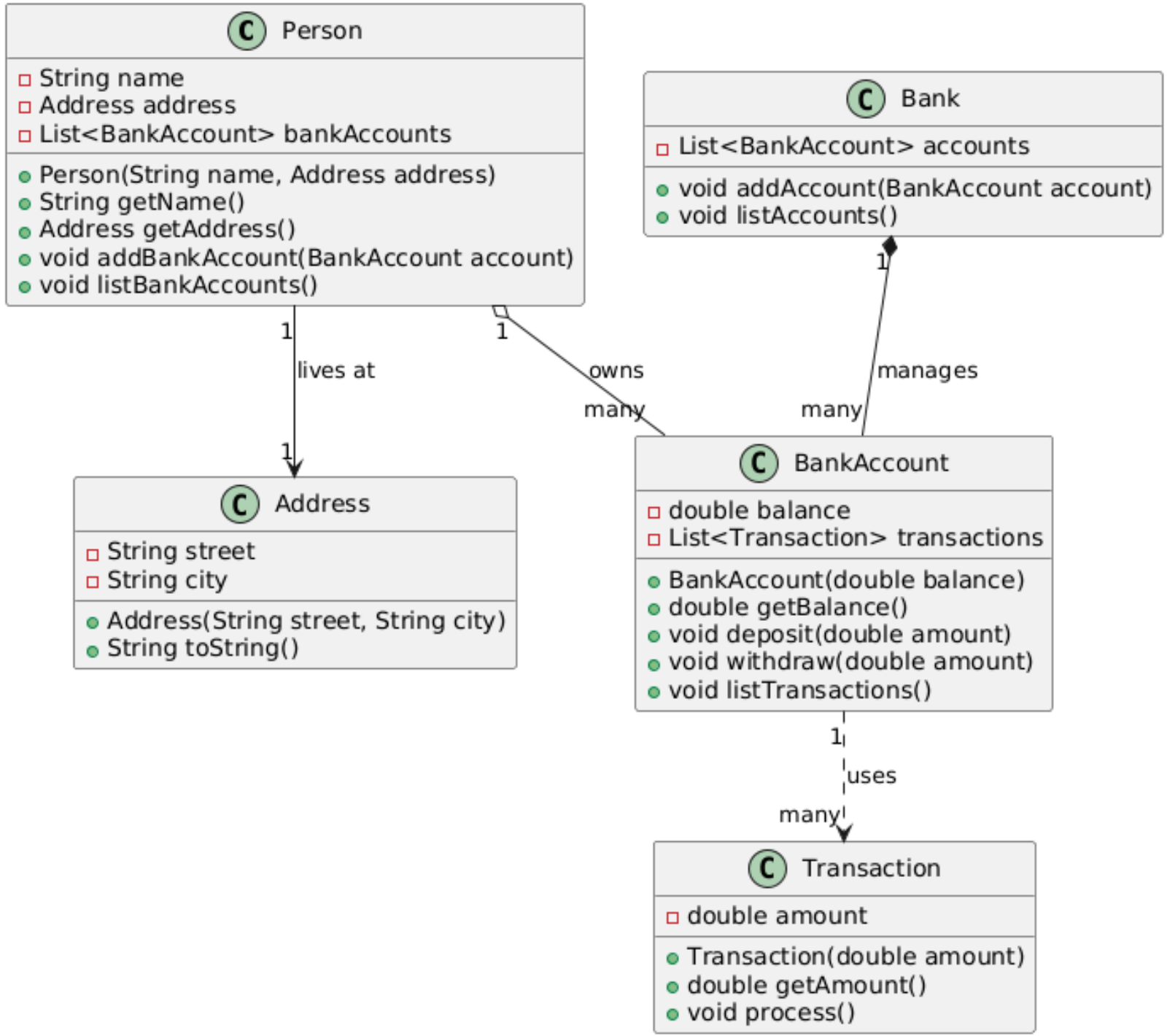
Strong has-a

A student will not be destroyed after a classroom is destroyed

They are stored somewhere else

Weak has-a




Person
String name
Address address
List<BankAccount> bankAccounts
Person(String name, Address address)
String getName()
Address getAddress()
void addBankAccount(BankAccount account)
void listBankAccounts()
Bank
List<BankAccount> accounts
void addAccount(BankAccount account)
void listAccounts()
1
lives at
1
1
owns
many
1
manages
many
Address
String street
String city
Address(String street, String city)
String toString()
BankAccount
double balance
List<Transaction> transactions
BankAccount(double balance)
double getBalance()
void deposit(double amount)
void withdraw(double amount)
void listTransactions()
1
uses
many
Transaction
double amount
Transaction(double amount)
double getAmount()
void process()